

DBMS – PRODUCT AND CATEGORY MANAGEMENT

Project Title: Myntra – E-Commerce database Management System Skill

1. Database Design

1.1 Category Table

Attribute	Data Type	Key	Description
Category_ID	INT	PK	Unique category ID
Category_Name	VARCHAR(50)	-	Category name
Category_Description	TEXT	-	Category description
Status	VARCHAR(20)	-	Active / Inactive

1.2 Product Table

Attribute	Data Type	Key	Description
Product_ID	INT	PK	Unique product ID
Product_Name	VARCHAR(100)	-	Product name
Category_ID	INT	FK	References Category
Seller_ID	INT	FK	References Seller
Brand	VARCHAR(50)	-	Brand
Size	VARCHAR(20)	-	Available size
Color	VARCHAR(30)	-	Product color
Price	DECIMAL(10,2)	-	Product price
Discount	DECIMAL(5,2)	-	Discount
Stock	INT	-	Available stock

2. Primary and Foreign Key Relationship

Category → Product

- Category.Category_ID → Primary Key
- Product.Product_ID → Primary Key
- Product.Category_ID → Foreign Key referencing Category.Category_ID
- Relationship: One-to-Many (1:M)

3. Table Creation

3.1 Create Category Table

```
SQL> CREATE TABLE Category (  
2     Category_ID          INT PRIMARY KEY,  
3     Category_Name        VARCHAR(50),  
4     Category_Description VARCHAR(200),  
5     Status               VARCHAR(20)  
6 );  
  
Table created.
```

3.2 Create Product Table

```
SQL> CREATE TABLE Product (  
2     Product_ID          INT PRIMARY KEY,  
3     Product_Name        VARCHAR(100),  
4     Category_ID         INT,  
5     Brand               VARCHAR(50),  
6     Product_Size        VARCHAR(20),  
7     Color               VARCHAR(30),  
8     Price               DECIMAL(10,2),  
9     Discount            DECIMAL(5,2),  
10    Stock               INT,  
11    CONSTRAINT fk_category FOREIGN KEY (Category_ID)  
12         REFERENCES Category(Category_ID)  
13 );  
  
Table created.
```

4. Data Insertion

4.1 Category Records

```
SQL> INSERT INTO Category VALUES (101, 'Men Clothing', 'Shirts, T-shirts, Jeans', 'Active');
1 row created.

SQL> INSERT INTO Category VALUES (102, 'Women Clothing', 'Dresses, Tops, Sarees', 'Active');
1 row created.

SQL> INSERT INTO Category VALUES (103, 'Footwear', 'Shoes, Sandals, Slippers', 'Active');
1 row created.

SQL> INSERT INTO Category VALUES (104, 'Watches', 'Analog and Digital watches', 'Active');
1 row created.

SQL> INSERT INTO Category VALUES (105, 'Bags', 'Backpacks, Handbags', 'Active');
1 row created.

SQL> INSERT INTO Category VALUES (106, 'Beauty', 'Makeup and Skincare', 'Active');
1 row created.

SQL> |
```

4.2 Product Records

```
SQL> INSERT INTO Product VALUES (201, 'Cotton Shirt', 101, 'Levis', 'M', 'Blue', 1299.00, 10.00, 50);
1 row created.

SQL> INSERT INTO Product VALUES (202, 'Floral Dress', 102, 'Vero Moda', 'S', 'Red', 1999.00, 15.00, 30);
1 row created.

SQL> INSERT INTO Product VALUES (203, 'Running Shoes', 103, 'Nike', '9', 'Black', 3499.00, 20.00, 40);
1 row created.

SQL> INSERT INTO Product VALUES (204, 'Analog Watch', 104, 'Fossil', 'Free', 'Brown', 5999.00, 5.00, 20);
1 row created.

SQL> INSERT INTO Product VALUES (205, 'Backpack', 105, 'Wildcraft', 'Free', 'Grey', 1799.00, 10.00, 35);
1 row created.

SQL> INSERT INTO Product VALUES (206, 'Lipstick', 106, 'Maybelline', 'Free', 'Pink', 499.00, 0.00, 100);
1 row created.
```

5. Data Retrieval

5.1 Display Category Records

```
SQL> Select * from Category;
```

```
CATEGORY_ID CATEGORY_NAME
```

```
CATEGORY_DESCRIPTION
```

```
STATUS
```

```
-----  
          101 Men Clothing  
Shirts, T-shirts, Jeans  
Active
```

```
          102 Women Clothing  
Dresses, Tops, Sarees  
Active
```

```
CATEGORY_ID CATEGORY_NAME
```

```
CATEGORY_DESCRIPTION
```

```
STATUS
```

```
-----  
          103 Footwear  
Shoes, Sandals, Slippers  
Active
```

```
          104 Watches  
Analog and Digital watches
```

```
CATEGORY_ID CATEGORY_NAME
```

```
CATEGORY_DESCRIPTION
```

```
STATUS
```

```
-----  
Active
```

```
          105 Bags  
Backpacks, Handbags  
Active
```

```
          106 Beauty
```

```
CATEGORY_ID CATEGORY_NAME
```

```
CATEGORY_DESCRIPTION
```

```
STATUS
```

```
-----  
Makeup and Skincare  
Active
```

```
6 rows selected.
```

5.2 Display Product Records

```
SQL> Select * from Product;
```

```
PRODUCT_ID
-----
PRODUCT_NAME
-----
CATEGORY_ID BRAND
-----
PRODUCT_SIZE      COLOR      PRICE  DISCOUNT
-----
      STOCK
-----
      201
Cotton Shirt
      101 Levis
```

```
PRODUCT_ID
-----
PRODUCT_NAME
-----
CATEGORY_ID BRAND
-----
PRODUCT_SIZE      COLOR      PRICE  DISCOUNT
-----
      STOCK
-----
M      50      Blue      1299      10
```

```
PRODUCT_ID
-----
PRODUCT_NAME
-----
CATEGORY_ID BRAND
-----
PRODUCT_SIZE      COLOR      PRICE  DISCOUNT
-----
      STOCK
-----
      202
Floral Dress
      102 Vero Moda
```

```
PRODUCT_ID
-----
PRODUCT_NAME
-----
CATEGORY_ID BRAND
-----
PRODUCT_SIZE      COLOR      PRICE  DISCOUNT
-----
      STOCK
-----
S      30      Red      1999      15
```

```
PRODUCT_ID
-----
PRODUCT_NAME
-----
CATEGORY_ID BRAND
-----
```

6. Product and category update

```
SQL> UPDATE Product
  2  SET Price = 1399.00, Discount = 12.00
  3  WHERE Product_ID = 201;

1 row updated.

SQL>
SQL> -- Update category status
SQL> UPDATE Category
  2  SET Status = 'Inactive'
  3  WHERE Category_ID = 103;

1 row updated.

SQL>
SQL> COMMIT;

Commit complete.
```

6.1 After update viewing the table

```
SQL> Select * from Product where Product_ID=201;
```

PRODUCT_ID	PRODUCT_NAME	CATEGORY_ID	BRAND	PRODUCT_SIZE	COLOR	PRICE	DISCOUNT
201	Cotton Shirt	101	Levis				

```
PRODUCT_ID
-----
PRODUCT_NAME
-----
CATEGORY_ID BRAND
-----
PRODUCT_SIZE      COLOR      PRICE      DISCOUNT
-----
STOCK
-----
201
Cotton Shirt
101 Levis

PRODUCT_ID
-----
PRODUCT_NAME
-----
CATEGORY_ID BRAND
-----
PRODUCT_SIZE      COLOR      PRICE      DISCOUNT
-----
STOCK
-----
M      Blue      1399      12
50
```

```
SQL> Select * from Category where Category_ID=103;
```

```
CATEGORY_ID CATEGORY_NAME
```

```
-----  
CATEGORY_DESCRIPTION
```

```
-----  
STATUS
```

```
-----  
          103 Footwear  
Shoes, Sandals, Slippers  
Inactive
```

7. Deletion

7.1 Product deletion

```
SQL> DELETE FROM Product  
2 WHERE Product_ID = 206;
```

```
1 row deleted.
```

```
SQL> Select * from Product where Product_ID=206;
```

```
no rows selected
```

7.2 Category deletion

```
SQL> DELETE FROM Category  
2 WHERE Category_ID = 106  
3 AND Category_ID NOT IN (SELECT Category_ID FROM Product);
```

```
1 row deleted.
```

```
SQL>
```

```
SQL> COMMIT;
```

```
Commit complete.
```

```
SQL> |
```

8.Inner Join

```
SQL> -- Inner Join: list all products with their category name
SQL> SELECT p.Product_ID, p.Product_Name, c.Category_Name, p.Price, p.Stock
  2 FROM Product p
  3 JOIN Category c ON p.Category_ID = c.Category_ID;
```

```
PRODUCT_ID
-----
PRODUCT_NAME
-----
CATEGORY_NAME                                PRICE      STOCK
-----
          201
Cotton Shirt
Men Clothing                                1399         50

          202
Floral Dress
Women Clothing                              1999         30
```

```
PRODUCT_ID
-----
PRODUCT_NAME
-----
CATEGORY_NAME                                PRICE      STOCK
-----

          203
Running Shoes
Footwear                                    3499         40

          204
Analog Watch
```

```
PRODUCT_ID
-----
PRODUCT_NAME
-----
CATEGORY_NAME                                PRICE      STOCK
-----

          205
Backpack
Bags                                        1799         35
```

9. Left join

```
SQL> -- Left Join: show all categories, even those with no products
SQL> SELECT c.Category_ID, c.Category_Name, p.Product_Name
  2 FROM Category c
  3 LEFT JOIN Product p ON c.Category_ID = p.Category_ID;
```

```
CATEGORY_ID CATEGORY_NAME
-----
PRODUCT_NAME
-----

          101 Men Clothing
Cotton Shirt

          102 Women Clothing
Floral Dress

          103 Footwear
Running Shoes

CATEGORY_ID CATEGORY_NAME
-----
PRODUCT_NAME
-----

          104 Watches
Analog Watch

          105 Bags
Backpack

SQL> |
```

10.Count operation

```
SQL> -- Count of products and total stock per category
SQL> SELECT c.Category_Name,
2         COUNT(p.Product_ID) AS Total_Products,
3         SUM(p.Stock)         AS Total_Stock,
4         AVG(p.Price)         AS Avg_Price
5 FROM Category c
6 JOIN Product p ON c.Category_ID = p.Category_ID
7 GROUP BY c.Category_Name
8 ORDER BY Total_Products DESC;
```

CATEGORY_NAME	TOTAL_PRODUCTS	TOTAL_STOCK
Men Clothing 1399	1	50
Women Clothing 1999	1	30
Bags 1799	1	35

CATEGORY_NAME	TOTAL_PRODUCTS	TOTAL_STOCK
Watches 5999	1	20
Footwear 3499	1	40

```
SQL> |
```

